UNIVERSIDADE DE BRASÍLIA
Faculdade UnB Gama - FGA
FGA0221 – Inteligência Artificial

PORTFÓLIO - PARTE 2

Métodos Clássicos de Inteligência Artificial

Autor	Gabriel Alves
Matrícula	241025523
Disciplina	FGA0221 – Inteligência Artificial
Professor	Fabiano Araujo Soares
Turma	T01 - 2º/2026
Data de entrega	09/10/2026

Sumário

Sumário **2**
1 Introdução **3**
1.1 Organização do Relatório 3
2 Projetos **4**
Projeto 1: Busca Informada 4
Descrição do Problema 4
Algoritmo Desenvolvido 4
Resultados 4
Considerações Finais 7
Projeto 2: Busca Não Informada 8
Descrição do Problema 8
Algoritmo Desenvolvido 8
Resultados 8
Considerações Finais 11
Projeto 3: Busca Complexa 12
Descrição do Problema 12
Algoritmo Desenvolvido 12
Resultados 13
Considerações Finais 15
Projeto 4: Algoritmo Genético 16
Descrição do Problema 16
Algoritmo Desenvolvido 16
Resultados 16
Considerações Finais 18
Projeto 5: CSPs — Satisfação de Restrições 19
Descrição do Problema 19
Algoritmo Desenvolvido 19
Resultados 19
Considerações Finais 20
Projeto 6: Banco de Conhecimento 21
Descrição do Problema 21
Algoritmo Desenvolvido 21
Resultados 21
Considerações Finais 23
3 Considerações Finais Gerais **24**
Referências Bibliográficas **24**

1 Introdução

Este relatório integra a **Parte 2 do Portfólio** da disciplina FGA0221 – Inteligência Artificial (2º/2026), conforme o Plano de Ensino do Prof. Fabiano Araujo Soares. O tema desta parte é **Métodos Clássicos de Inteligência Artificial**, cobrindo técnicas fundamentais de busca, algoritmos evolucionários, satisfação de restrições (CSP) e sistemas baseados em conhecimento.

Cada um dos seis projetos a seguir é acompanhado de: (i) descrição do problema, (ii) descrição detalhada do algoritmo desenvolvido, (iii) resultados com imagens e valores e (iv) considerações finais. Os arquivos Python comentados são entregues em conjunto, nomeados conforme indicado ao início de cada seção.

1.1 Organização do Relatório

O relatório está estruturado em seis seções principais, uma por projeto:

- Projeto 1 - Busca Informada (arquivo: proj1_busca_informada.py)
- Projeto 2 - Busca Não Informada (arquivo: proj2_busca_nao_informada.py)
- Projeto 3 - Busca Complexa (arquivo: proj3_busca_complexa.py)
- Projeto 4 - Algoritmo Genético (arquivo: proj4_algoritmo_genetico.py)
- Projeto 5 - CSPs (arquivo: proj5_csp.py)
- Projeto 6 - Banco de Conhecimento (arquivo: proj6_banco_conhecimento.py)

2 Projetos

Projeto 1: Busca Informada

Descrição do Problema

O problema consiste no planejamento de rota para um drone de carga incumbido de transportar encomendas do campus da UnB até o Autódromo Internacional de Brasília, a aproximadamente 4,9 km a oeste. O trajeto cruza uma área urbana real contendo **aeródromos, helipontos e estabelecimentos de saúde**, cujas áreas circundantes definem zonas de sobrevoo proibido (NFZs). Três aspectos motivam a formulação desse cenário: (i) a aplicação prática de busca em grafos para *pathfinding* sobre malhas viárias reais, superando simplificações sintéticas; (ii) a inclusão do Autódromo Internacional (equipado com heliponto registrado na ICA 100-40), o que introduz restrições regulatórias autênticas; e (iii) a adoção do **tempo de voo** como métrica principal de custo, em vez da distância física, justificando a investigação de heurísticas informadas para além da busca cega por menor caminho.
As restrições de voo são tratadas de forma **estrita**: arestas que interceptam zonas interditadas são excluídas do grafo, evitando a expansão desnecessária em estados inviáveis e garantindo a conformidade regulatória das rotas geradas.

Algoritmo Desenvolvido

Para a construção da base de dados espacial, empregou-se a API OSM 0.6 devido à sua superior estabilidade em relação ao serviço Overpass. A área de estudo (~4,8 × 2,7 km) foi particionada em três faixas para contornar a limitação de 50.000 nós por requisição; como a API retorna vias integrais, a fusão dos dados por identificadores preservou a continuidade topológica da rede, totalizando 67.324 nós e 2.692 vias.

A partir dessa base, modelou-se um grafo não direcionado representando a malha viária navegável, desconsiderando vias de pedestres ou de uso restrito. O custo de cada aresta foi definido pelo tempo de voo, calculado a partir do comprimento do segmento e da velocidade limite associada à classe da via (limitada a 80 km/h), resultando em um grafo inicial composto por 11.140 nós e 12.310 arestas.

As restrições de sobrevoo foram incorporadas agrupando-se 28 pontos de interesse (aeródromos, helipontos e estabelecimentos de saúde) por proximidade espacial (250 m), definindo 10 zonas de interdição com raios operacionais de 600 m para infraestruturas aeronáuticas e 200 m para unidades de saúde. Para garantir a eficácia do bloqueio, aplicou-se amostragem geométrica ao longo das arestas a cada 30 m, eliminando 2.741 segmentos que interceptavam as zonas interditadas (22,3% do total) e consolidando um grafo viável com 8.555 nós e 9.404 arestas.

O cenário experimental foi estabelecido entre o ponto mais próximo ao campus da UnB e o nó mais distante em linha reta (4.918 m), maximizando a complexidade do percurso. Avaliou-se o desempenho dos algoritmos Dijkstra, A* com heurística geométrica admissível, Greedy Best-First e variações do A* ponderado ($W \in \{1{,}5; 2; 3\}$), além da heurística perfeita h*(n) obtida via Dijkstra reverso para delimitar o limite teórico de eficiência da busca informada.

Resultados

As restrições mudam a resposta. A rota ótima *sem* restrições custa **481,3 s** e **sobrevoa 3 zonas interditadas** (1 aeródromo e 2 hospitais). Com as restrições, o custo ótimo sobe para **528,9 s (+9,9%)** e o caminho é diferente. A restrição, portanto, não é decorativa: ela realmente altera a solução.

Algoritmo	Custo (s)	Nós expandidos	Rel. Dijkstra	Fronteira	Gap (%)	Tempo (s)
Dijkstra	528,9	8.551	100,0 %	108	0,00	0,0551
A* (h geométrica)	528,9	7.689	89,9 %	125	0,00	0,0679
A* (h* perfeita)	528,9	**261**	**3,1 %**	113	0,00	0,0022
Greedy Best-First	732,8	981	11,5 %	120	38,56	0,0085
A* ponderado (W=1,5)	528,9	5.425	63,4 %	168	0,00	0,0537
A* ponderado (W=2)	532,5	2.065	24,2 %	151	0,67	0,0201
A* ponderado (W=3)	532,5	1.149	13,4 %	131	0,67	0,0087

Três leituras merecem destaque:

1. **A heurística geométrica é fraca.** A* expande 7.689 nós contra 8.551 do Dijkstra, apenas **10,1 % de redução**, apesar de ser mais lento (0,068 s vs 0,055 s). A causa está medida: a razão mediana h/h* é **0,445** (p90 = 0,507; mínimo = 0,181; n = 6.961). Uma heurística que vale ~44 % do ótimo não orienta muito. A malha é regular, "em grade", mas o custo é tempo por hierarquia viária, então a distância reta ignora as vias rápidas.
2. **Uma boa heurística vale uma ordem de grandeza.** Com h* perfeita, o A* expande **261 nós: 32,8× menos que o Dijkstra** e 4,6 % do tempo. Esse número estabelece o **teto do que qualquer heurística admissível poderia alcançar**, e é o argumento central a favor de investir em heurística de domínio: um modelo de vento, ou um ponderamento que privilegie autoestradas, poderia capturar boa parte desse ganho.
3. **Inadmissibilidade custa qualidade.** Greedy Best-First expande só 981 nós (11,5 %), mas entrega solução **38,6 % pior** que a ótima. A* ponderado com W=2 e W=3 é o melhor compromisso: **75,8 % e 86,6 % menos expansões** por apenas **0,67 %** de gap.

A rota ótima tem **261 arestas, 7.188 m de comprimento e 528,9 s**: razão desvio/reta de 1,46, coerente com contornar as zonas.

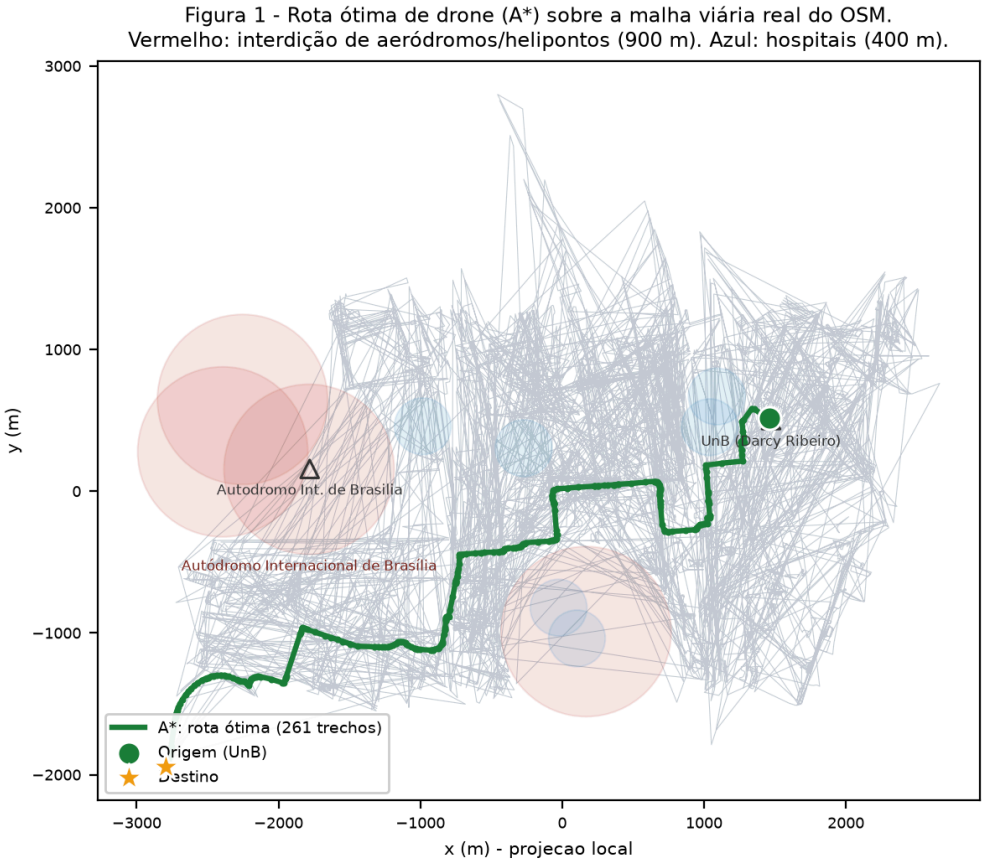


Figura 1 - Malha de voo sobre a área operacional, zonas de interdição e rota ótima do A.*

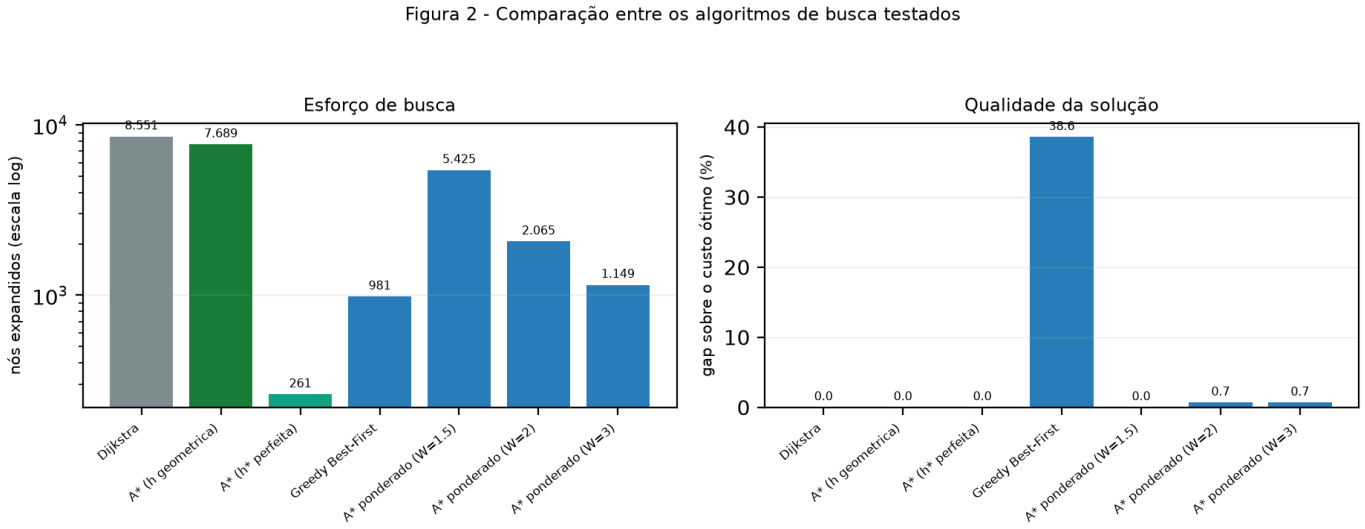


Figura 2 - Esforço de busca (escala log) e gap sobre o custo ótimo.

Sensibilidade aos raios. Como os raios são parâmetros arbitrados, o experimento foi repetido em três cenários:

Cenário	Raios (m)	Arestas removidas	Nós viáveis	Custo ótimo (s)	Redução A* vs Dijkstra
Permissivo	400 / 150	12,5 %	9.659	528,9	8,4 %
Adotado	600 / 200	22,3 %	8.555	528,9	10,1 %
Restritivo	900 / 350	44,7 %	2.429	208,2	75,0 %

O cenário restritivo merece cuidado: o custo ótimo **cai** para 208,2 s, o que parece uma melhoria, mas é um **artefato de inviabilidade**. Com 44,7 % das arestas removidas a malha se fragmenta (4.884 componentes), e o destino é forçado a recuar para 2.340 m da origem. Resta 2.429 nós, um subgrafo isolado. O ANTERIOR ALCANCE DEIXA DE EXISTIR: o que resta é a mesma região, com menos destinos, e por isso "mais barato". Isso demonstra que **viabilidade vem antes de custo**: a optimalidade só tem sentido num grafo conectado que contenha o destino real, e comparar custos entre cenários com alcances diferentes é inválido.

Considerações Finais

Em síntese, os experimentos realizados não apenas validam a eficácia teórica do algoritmo A* frente à busca cega por menor caminho, mas também explicitam em termos numéricos as condições sob as quais a incorporação de heurísticas se justifica. No domínio analisado, a heurística baseada na distância geométrica em linha reta (apresentando uma razão mediana h/h* de apenas 0,445) revelou-se insuficiente para contornar a malha viária urbana e as restrições operacionais. Todavia, a comparação direta com a heurística perfeita h* transforma uma simples intuição sobre busca informada em uma meta quantitativa de engenharia, demonstrando que o grande gargalo de desempenho reside no refinamento da modelagem do conhecimento do domínio, e não na capacidade computacional do algoritmo em si.

Apesar dos resultados promissores, a abordagem proposta apresenta limitações simplificadoras que devem ser consideradas em futuras iterações. Primeiramente, a definição dos raios de sobrevoo proibido adota distâncias operacionais genéricas que, embora plausíveis, não substituem o zoneamento regulatório individualizado pela ANAC. Além disso, a premissa de velocidade fixa por hierarquia de via ignora fatores dinâmicos cruciais, como correntes de vento que são de especial relevância no cenário de Brasília, massa da carga útil e consumo de bateria, reduzindo o modelo a um plano de rotas bidimensional estático. Essa abstração também desconsidera a presença de obstáculos urbanos em três dimensões, tais como edifícios e linhas de alta tensão, bem como os sentidos únicos de circulação viária, cuja observância seria excessivamente conservadora para operações aéreas.

Para superar tais restrições, trabalhos futuros podem explorar o desenvolvimento de heurísticas de domínio que incorporem estimativas de velocidade efetiva ponderadas pelas vias da malha, aproximando o desempenho do A* ao limite ideal. Paralelamente, a formulação do custo pode ser expandida para cenários multiobjetivo equilibrando tempo, segurança e autonomia energética, bem como para a conversão de zonas de exclusão estritas em penalidades graduais de risco. Por fim, ressalta-se a alta generalização da estrutura implementada, que se mantém agnóstica ao domínio de aplicação e pode ser prontamente adaptada para o planejamento de rotas em navegação robótica ou otimização de fluxos em redes complexas.

Projeto 2: Busca Não Informada

Descrição do Problema

Quando executamos o comando de instalação de um pacote como o JupyterLab, o gerenciador de dependências precisa identificar com precisão quais bibliotecas adicionais são necessárias e em qual sequência elas devem ser preparadas. Não é suficiente apenas saber que um pacote depende de outro, é fundamental navegar por toda a árvore de dependências até a sua base para montar um plano de execução coerente, garantindo que cada requisito seja instalado antes daqueles que precisam dele.

Esse cenário é ideal para a aplicação de algoritmos de busca não informada. Primeiro, porque o sistema opera sem conhecimento prévio do todo, descobrindo as dependências de um pacote apenas ao ler seus metadados, sem uma função para estimar quanto falta para concluir o processo.

Segundo, porque a estrutura do problema forma um grafo direcionado acíclico cuja ordenação é a chave da solução. Como muitos pacotes são compartilhados entre diferentes dependentes, a ordem em que são descobertos não corresponde diretamente à sequência correta de instalação, o que torna a criação do plano o ponto central do desafio.

Para testar essa abordagem em diferentes escalas, foram analisados cinco pacotes do ecossistema Python: JupyterLab (com 80 pacotes), Notebook (81), Gradio (52), Poetry (40) e FastAPI (11).

Algoritmo Desenvolvido

Em relação aos dados, as informações sobre os requisitos de cada biblioteca foram obtidas por meio da API do PyPI. Para garantir a precisão da análise, os marcadores de ambiente foram validados para considerar apenas as dependências compatíveis com o sistema operacional em execução, evitando incluir pacotes desnecessários. Além disso, as dependências opcionais foram descartadas para refletir o comportamento padrão de instalação. Todas as respostas foram armazenadas localmente para permitir execuções subsequentes sem dependência de rede.

Na modelagem do grafo, uma conexão entre dois pacotes indica dependência direta, significando que o requisito deve ser instalado antes da biblioteca principal. É preciso atenção a esse sentido, pois a ordenação topológica tradicional inverteria a sequência ideal de instalação se não fosse tratada adequadamente.

Para resolver o problema, foram implementados quatro algoritmos distintos. A busca em largura (BFS) explora a estrutura por níveis a partir da raiz. A busca em profundidade (DFS) iterativa organiza os pacotes em pós-ordem para definir a sequência de instalação e identificar possíveis dependências circulares de forma eficiente. A busca em profundidade iterativa (IDDFS) testa gradualmente limites de profundidade para encontrar o menor número de etapas necessárias para instalar todo o conjunto. Por fim, a busca de custo uniforme (UCS) explora o espaço de estados considerando o custo de consulta, limitando as opções apenas aos pacotes já descobertos.

Para garantir a consistência das soluções, toda ordem gerada foi validada de forma independente em relação às dependências do conjunto e reavaliada a partir das folhas do grafo para determinar a distribuição ideal das etapas de instalação.

Resultados

O cenário principal foi executado com o pacote jupyterlab, cujo grafo de dependências contém 80 pacotes e 139 arestas, sem ciclos.

Algoritmo	Consultas	Pico de fronteira/pilha	Ondas	Ordem válida	Tempo (s)
BFS	80	32 (fronteira)	10	sim	0,0011

Algoritmo	Consultas	Pico de fronteira/pilha	Ondas	Ordem válida	Tempo (s)
DFS	80	8 (pilha)	10	sim	0,0002
IDDFS	80	10 (limite)	10	sim	0,0045
UCS	—	961.048 (fronteira)	—	incompleto	14,76

Os resultados de BFS, DFS e IDDFS são equivalentes nos aspectos principais. Os três algoritmos percorrem todo o conjunto de dependências, produzem uma ordem de instalação válida, realizam exatamente 80 consultas e identificam as 10 ondas mínimas de instalação. Esses resultados também foram verificados por meio de testes com assert, confirmando que os requisitos foram atendidos.
O número de consultas realizadas pelos três algoritmos é mínimo. Isso ocorre porque cada pacote precisa ser consultado pelo menos uma vez para que suas dependências sejam conhecidas. Sem essa consulta, não seria possível distinguir um pacote sem dependências de outro que depende de pacotes adicionais, o que poderia resultar em um plano de instalação incorreto. Como o grafo possui 80 pacotes e os três algoritmos realizam exatamente 80 consultas, eles atingem o limite inferior de ($|V| = 80$). Portanto, nesse aspecto, nenhuma busca mais sofisticada reduziria o número de requisições à rede.
Um dos resultados mais relevantes é a diferença entre o nível da BFS e o número mínimo de ondas de instalação. Embora ambos estejam relacionados à estrutura do grafo, eles representam medidas distintas. A BFS identifica o menor caminho entre a raiz e cada pacote, enquanto o cálculo das ondas de instalação considera a cadeia mais longa de dependências necessária para determinar quando cada pacote pode ser instalado. No caso do jupyterlab, a BFS identifica seis níveis, mas a análise das dependências demonstra que são necessárias dez ondas. Utilizar diretamente o nível da BFS resultaria em uma subestimação de quatro ondas, equivalente a 40% do total necessário.
A comparação entre os cinco pacotes analisados evidencia essa diferença:

Raiz	Pacotes	Arestas	Ondas mínimas	Nível BFS	Subestimação
jupyterlab	80	139	10	6	+4
notebook	81	145	11	6	+5
gradio	52	86	7	5	+2
poetry	40	51	6	6	0
fastapi	11	17	4	4	0

Os resultados mostram que a diferença entre essas medidas depende da estrutura de cada grafo. Nos casos de poetry e fastapi, os valores coincidem. Se apenas esses dois cenários fossem considerados, seria possível concluir equivocadamente que o nível da BFS constitui uma boa estimativa do número de ondas de instalação.
Essa diferença está relacionada ao compartilhamento de dependências. Quando um pacote é exigido por vários outros, ele pode ser alcançado por caminhos de comprimentos distintos. No conjunto analisado, 23 pacotes são compartilhados por mais de um dependente. Assim, um pacote

pode aparecer em um nível baixo da BFS por ser alcançável por um caminho curto, embora também faça parte de uma cadeia mais longa de dependências. Como a BFS registra a menor distância a partir da raiz, ela não representa, por si só, a profundidade máxima necessária para a instalação.
No caso do jupyterlab, a distribuição dos pacotes pelas dez ondas é de 43, 18, 7, 3, 2, 1, 1, 1, 3 e 1 pacote, respectivamente. Isso significa que 43 dos 80 pacotes, aproximadamente 54%, podem ser instalados na primeira onda, pois não possuem dependências dentro do conjunto analisado. A grande quantidade de pacotes que podem ser instalados inicialmente ajuda a explicar por que o grafo apresenta uma cadeia máxima relativamente curta em comparação com o número total de pacotes.

Figura 2 - Busca não informada: memória da busca e o erro do nível BFS

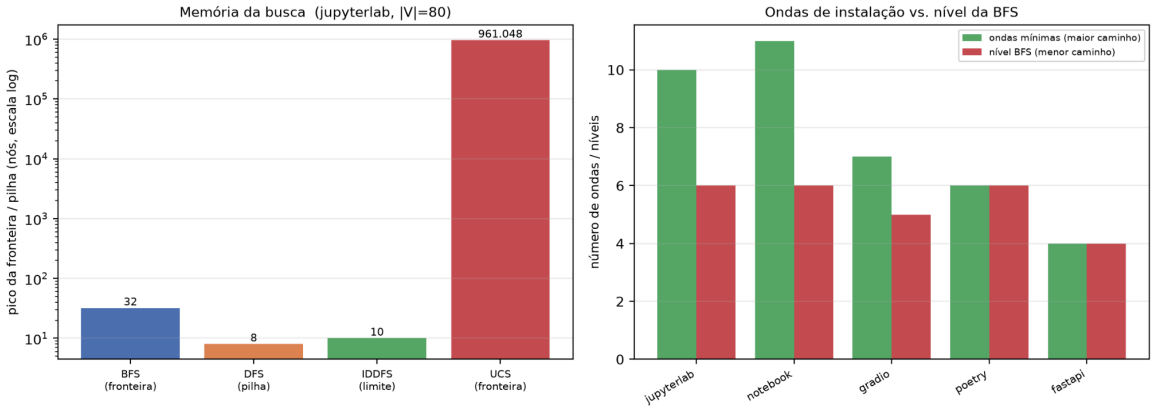


Figura 4 - À esquerda, o pico de memória da busca (escala log): a BFS guarda a fronteira inteira, a DFS só a pilha. À direita, ondas mínimas contra nível da BFS.

O UCS é exponencial, isso foi medido, não afirmado. O espaço de estados é o conjunto de subconjuntos de pacotes, de tamanho 2^|V|. A restrição de ações à fronteira reduz muito o número de estados alcançáveis, mas não elimina a explosão:

Raiz	|V|	2^|V| (teórico)	Estados expandidos	Resultado
fastapi	11	$2{,}05 \times 10^{3}$	319	completo (0,0038 s)
poetry	40	$1{,}1 \times 10^{12}$	200.001	**incompleto**
gradio	52	$4{,}5 \times 10^{15}$	200.001	**incompleto**
jupyterlab	80	$1{,}21 \times 10^{24}$	200.001	**incompleto**
notebook	81	$2{,}42 \times 10^{24}$	200.001	**incompleto**

O comportamento do algoritmo UCS (Uniform Cost Search) evidencia o crescimento do custo computacional à medida que o número de pacotes aumenta. O espaço teórico de estados corresponde ao conjunto de todos os subconjuntos possíveis de pacotes, cuja quantidade é dada por $(2^{|V|})$. Embora a restrição das ações aos pacotes disponíveis na fronteira reduza o número de estados alcançáveis, ela não impede a explosão combinatória.

O UCS concluiu a busca em apenas um dos cinco cenários, o fastapi. Nos outros quatro, o limite de 200.001 estados expandidos não foi suficiente para concluir a execução. No cenário principal, o algoritmo levou aproximadamente 14,76 segundos e acumulou 961.048 estados na fronteira, exigindo uma quantidade considerável de memória em comparação com o tamanho do grafo original.
Esses resultados não significam que o UCS seja inadequado para qualquer problema de busca. A limitação está na sua aplicação a esta formalização. Como todas as ações possuem custo uniforme, o algoritmo não oferece uma vantagem prática na otimização das consultas: a BFS e a DFS já atingem o número mínimo possível, comprovado pelo limite de uma consulta por pacote. Dessa forma, o custo adicional do UCS não produz uma redução no número de consultas e torna sua utilização desnecessária para esse objetivo específico.
Nenhum dos cinco grafos reais analisados apresentou dependências circulares. Portanto, foram identificados zero ciclos no conjunto de dados. Esse resultado é compatível com a expectativa de que os pacotes publicados no PyPI não apresentem ciclos de dependências que inviabilizem a resolução normal de suas instalações. Para verificar se a ausência de ciclos era uma característica dos dados, e não uma falha do detector, também foi realizado um teste com um grafo sintético contendo o ciclo (a \rightarrow b \rightarrow c \rightarrow a). O detector identificou corretamente o percurso e retornou ['a', 'b', 'c', 'a'], confirmando que o mecanismo reconhece esse tipo de estrutura.

Considerações Finais

O projeto mostra que **escolher o algoritmo errado custa mais que escolher o algoritmo lento**. No mesmo problema, BFS, DFS e IDDFS resolvem em ~1 ms, enquanto o UCS falha após 15s e 200 mil estados não por ser mais lento, mas por ser inaplicável. E mostra que a comparação entre algoritmos precisa de uma **verificação independente**: sem a relaxação a partir das folhas, a tabela teria relatado 6 ondas (a resposta da BFS) como se fosse verdade.
Limitações: (i) A resolução ignora **conflitos de versão**: o grafo não distingue attrs>=23 de attrs<21, e um resolvedor real precisaria escolher versões compatíveis este é o ponto em que o problema deixaria de ser busca cega e passaria a exigir backtracking. (ii) Extras, wheels por plataforma e dependências opcionais não são modeladas. (iii) Os pacotes são instalados em série; o modelo de ondas paralelas supõe que instalar um pacote não interfere no outro. (iv) O cache congela o estado do PyPI na data da coleta, então o fecho pode mudar com o tempo.
Melhorias: (i) Acrescentar resolução de versões com retrocesso, transformando o problema em busca com Backup e poda por versão incompatível. (ii) Substituir o UCS por busca em largura exaustiva ou por **eliminação de grau de entrada zero** (Kahn), que é O(|V| + |E|) e é o que o pip faz de fato. (iii) Paralelizar as ondas, já que 43 dos 80 pacotes são instaláveis simultaneamente. (iv) Explorar o IDDFS em paralelo aos demais: é o único dos três que **prova** a minimalidade, em vez de apenas supô-la.
Generalização: A mesma estrutura resolve ordenação de tarefas de build (make, CMake), planejamento de tarefas encadeadas em workflows, e qualquer problema de "descobrir dependências desconhecidas e produzir ordem de execução válida" inclusive sem grafo prévio, onde cada aresta só é confirmada quando o nó correspondente é consultado.

Projeto 3: Busca Complexa

Descrição do Problema

O problema nasce de um pedido comum a qualquer serviço de música: montar uma playlist que siga um perfil. O usuário informa a duração aproximada, os valores médios que quer para alguns atributos (**energia**, **positividade**, **acústica**, **dançabilidade** e **andamento**) e o formato que a energia deve desenhar ao longo da lista, que pode ser constante, crescente, decrescente ou com pico no meio. O programa precisa escolher quais faixas entram e em que ordem, e é essa junção de seleção com ordenação que afasta o problema de uma simples filtragem por compatibilidade.
O espaço de busca explica por que filtros não bastam. Uma playlist é uma permutação de um subconjunto de cem faixas, e o número de candidatos explode para além de qualquer enumeração. A paisagem ainda é cheia de ótimos locais: playlists parecidas têm qualidade parecida, e nada garante que o melhor vizinho de uma boa solução conduza à solução ótima. São exatamente as condições do capítulo 4 de Russell e Norvig (2022), dedicado à busca em ambientes complexos, e por isso o projeto aplica Hill Climbing e Simulated Annealing no lugar de buscas exaustivas.
A escolha de dados reais também faz parte do problema. Em vez de gerar canções sintéticas, as cem faixas vieram do catálogo público do Spotify, com metadados e atributos numéricos medidos, o que dá ao exercício o caráter de um sistema que poderia ser usado de verdade.

Algoritmo Desenvolvido

Dados: A coleta usou o endpoint de busca da API do Spotify em modo client credentials, com consultas por gênero e ano no mercado brasileiro, até reunir cem faixas com nome, artistas, álbum, ano, duração e código ISRC. O endpoint nativo de atributos de áudio está bloqueado para aplicativos novos desde novembro de 2024 e responde HTTP 403, então as características numéricas vieram da API FreqBlog, um serviço que reproduz o formato do endpoint antigo e aceita consulta por ISRC. Os campos usados são energia, valência, acústica, dançabilidade, andamento, tom, modo e volume. Instrumentalidade, presença de voz e presença de público vieram nulos em todas as faixas e ficaram de fora do perfil, decisão registrada nos metadados do dataset. O resultado da coleta é um único arquivo JSON que o programa apenas lê; não existe chamada de rede nem chave de API no código. Uma cópia do arquivo fica disponível na nuvem, no endereço apontado pela constante URL_DATASET_NUVEM dos códigos dos projetos 3, 4 e 6, para o caso de a entrega não acompanhar o arquivo local.

Representação: Faixa e Perfil são dataclasses imutáveis, e uma playlist é simplesmente uma lista ordenada de faixas. A ordem importa porque dois componentes da qualidade, a progressão e as transições, dependem da vizinhança entre posições. O perfil aceita alvos opcionais: o usuário pode ignorar qualquer atributo, e a função de compatibilidade deixa de cobrá-lo.

Função de qualidade: A nota de uma playlist é a soma ponderada de cinco componentes, todos entre 0 e 1. A compatibilidade, com peso 0,30, mede a distância média entre os valores de cada faixa e os alvos pedidos, normalizando o andamento por 100 BPM para igualar as escalas. A duração, com peso 0,20, decai exponencialmente com o erro absoluto em relação ao alvo, usando constante de 180 segundos. A progressão, com peso 0,25, compara a sequência de energias com a curva ideal do padrão escolhido depois de normalizar as duas para o intervalo entre 0 e 1, o que torna a nota imune ao nível médio da playlist e sensível apenas ao formato. As transições, com peso 0,15, calculam a média de 1 menos a diferença de energia entre faixas vizinhas, punindo saltos bruscos. A diversidade, com peso 0,10, parte da proporção de artistas distintos e desconta 0,1 por cada par consecutivo que compartilha artista.

Vizinhança: A partir de uma playlist são gerados 20 vizinhos sorteados entre três movimentos: trocar duas posições, substituir uma faixa por outra do catálogo que não esteja na lista e inverter o trecho entre dois cortes. Cada vizinho é uma cópia nova, nunca a mesma referência. Os três movimentos serão reaproveitados como mutação no algoritmo genético do projeto 4.

Buscas: O Hill Climbing segue o esqueleto da aula 08: avalia todos os vizinhos e move para o melhor apenas quando ele supera o atual, parando no primeiro ótimo local. Para contornar os

ótimos locais, cada execução parte de uma playlist aleatória e o experimento guarda o melhor resultado de cinco reinícios. O Simulated Annealing sorteia um vizinho por passo e aceita pioras com probabilidade exp(delta/T), com a temperatura caindo por multiplicação; o histórico registra o melhor fitness visto até o momento, e não o do estado corrente, porque a busca pode estar temporariamente num ponto pior.

Calibração do SA: Os parâmetros sugeridos no material da aula, temperatura inicial 50, resfriamento 0,95 e mínimo 0,01, se mostraram ruins para esta função de qualidade, que varia em passos de centésimos. Com temperatura 50 a probabilidade exp(delta/T) fica colada em 1 e o algoritmo aceita quase qualquer piora, gastando a maior parte dos seus 166 passos num passeio aleatório. Uma varredura de nove configurações sobre os três perfis do experimento, documentada em results/proj3_ajuste_sa.md (Gdevalves, 2026), mostrou que o fator decisivo é o resfriamento lento, e não a temperatura inicial: toda a família com taxa 0,99 se igualou ao Hill Climbing. A configuração adotada foi temperatura inicial 0,5, resfriamento 0,99 e mínimo 0,001, que rende 618 passos.

Baselines: Dois limites de referência completam a comparação. A playlist aleatória embaralha o catálogo e preenche até estourar a duração. A gulosa escolhe sempre a faixa mais compatível que ainda cabe e só então ordena o resultado pela energia no sentido do padrão pedido.

Resultados

A tabela resume a média de cinco execuções de cada abordagem nos três perfis testados. Nos dois primeiros perfis o usuário fixou valores de energia, positividade, acústica ou dançabilidade e de andamento; no terceiro deixou todos os atributos em branco e pediu apenas a queda de energia.

Perfil	Aleatória	Gulosa	Hill Climbing	Simulated Annealing
30 min, alvos de atributo, crescente	0,830	0,825	0,892	**0,916**
60 min, alvos de atributo, pico	0,815	0,904	**0,910**	0,907
15 min, só decrescente	0,804	0,831	0,970	**0,981**

As duas buscas locais superam os baselines em todos os perfis, com vantagem modesta nos dois primeiros e uma diferença enorme no terceiro, onde a ordenação é o problema inteiro e a aleatória chega a 0,804 contra 0,981 do SA. A precisão na duração é outro sinal de que a função de qualidade funciona como guia: as playlists vencedoras fecharam em 30:02, 59:59 e 15:03 para alvos de 30, 60 e 15 minutos.

A playlist vencedora do perfil decrescente merece um olhar, por ilustrar o tipo de decisão que o fitness produz. Ela abre com Linkin Park em Numb (ao vivo, 2026), passa por Heavy Is the Crown, desce para Mega Hair de Netto Brito e Luan Pereira, volta a Linkin Park em Somewhere I Belong e encerra com Enemy, de Imagine Dragons com JID. São cinco faixas cujas energias caem degrau a degrau do início ao fim, totalizando 15:03. A repetição do Linkin Park custa uma penalidade de diversidade, compensada pela suavidade da descida, um equilíbrio que nenhum filtro por compatibilidade alcançaria sozinho.

O desempenho do guloso também conta uma história. Ele é forte quando o perfil pede atributos: no perfil de 60 minutos chega a 0,904, quase colado nas buscas locais, porque maximizar compatibilidade é exatamente o que ele faz. Mas é cego à ordem e determinístico, devolve sempre a mesma nota, e desaba para 0,831 quando o padrão de progressão vira o critério dominante. Essa lacuna é a razão de existir do problema.

A calibração do Simulated Annealing, com dez execuções por célula e os mesmos pontos de partida dos dois algoritmos, está resumida abaixo:

Configuração do SA	**Passos**	**Média geral**
50 / 0,95 / 0,01 (aula 08)	166	0,896
0,5 / 0,99 / 0,001 (adotada)	618	0,932
Hill Climbing (referência)	-	0,924

Com os valores da aula o SA ficava um degrau abaixo do Hill Climbing, 0,896 contra 0,924. A configuração adotada o deixa praticamente empatado com ele, 0,932 contra 0,924, com uma leve vantagem nos perfis com alvos e praticamente o mesmo resultado no perfil só de progressão. O custo é aceitável: cerca de 0,15 segundo por execução contra 0,01 do Hill Climbing, porque o SA avalia um vizinho por passo enquanto o HC avalia vinte.

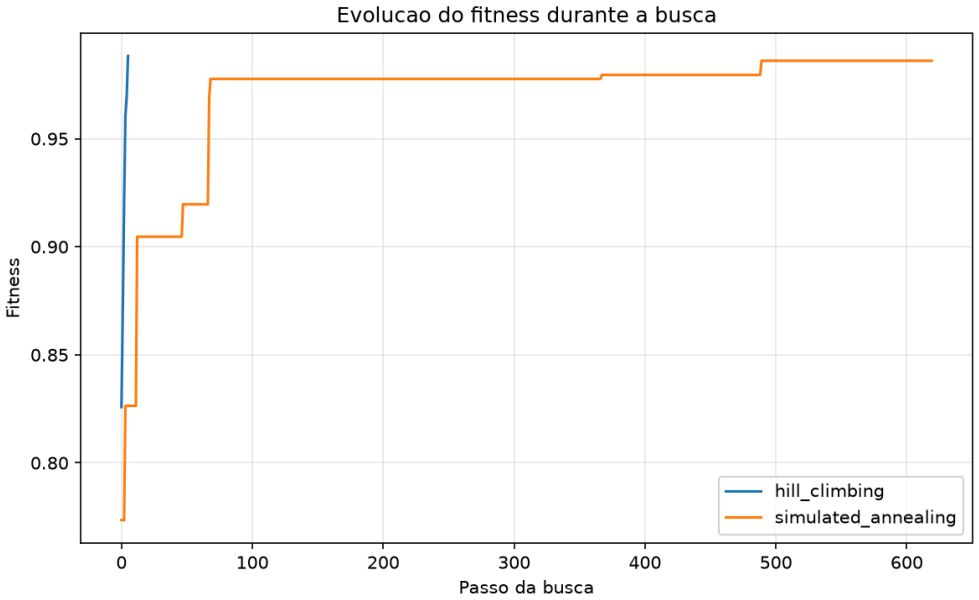


Figura 5 - evolução do fitness na melhor execução de cada busca local no perfil decrescente de 15 minutos. A curva do Simulated Annealing registra o melhor valor visto até o momento, por isso nunca desce.

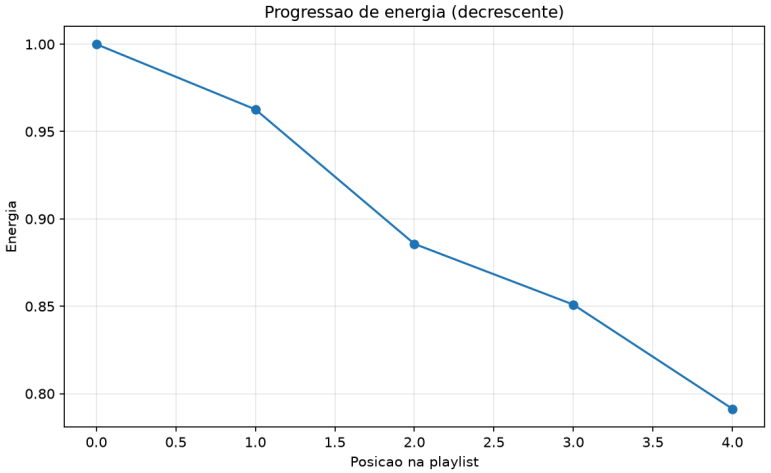


Figura 6 - energia ao longo da melhor playlist encontrada para o perfil decrescente, em queda constante da primeira à última faixa.

Considerações Finais

O projeto mostra que uma busca local guiada por uma função de qualidade bem desenhada resolve, em centésimos de segundo, um problema de combinação que filtros simples não alcançam, e que o Simulated Annealing, uma vez calibrado, empata com o Hill Climbing a um custo modesto. A calibração também ensina a lição mais valiosa do exercício: parâmetros de busca não são universais. Os valores herdados da aula foram pensados para uma função polinomial que varia em dezenas; numa função que varia em centésimos, a mesma temperatura inicial transforma o SA em quase sorteio.

As limitações são honestas. A função de qualidade é uma combinação linear de componentes arbitrados à mão, e a resposta final é tão boa quanto esses pesos, que seguem inexplorados em termos de sensibilidade. A normalização min-max da progressão amplifica ruído em playlists quase constantes, fazendo uma oscilação de centésimos parecer uma montanha-russa. Os atributos vêm da FreqBlog, que usa medidas próprias e não os classificadores originais do Spotify, com três campos ausentes, e o dataset é uma fotografia do catálogo na data da coleta. Por fim, o SA é determinístico apenas dentro de uma execução: a ordem dos conjuntos no Python depende do hash de strings, aleatório entre processos, o que muda o sorteio do movimento de substituição. A varredura de calibração foi feita com PYTHONHASHSEED fixo justamente para eliminar essa fonte de ruído, e quem reproduzir os números do relatório deve fazer o mesmo.

Como melhorias, o erro de duração poderia ser assimétrico, com estouro punido mais duramente que falta, e a progressão poderia ser comparada por correlação de Pearson em vez de erro quadrático após normalização. A vizinhança ganharia movimentos de inserção e remoção de faixas, que hoje não existem, e um estudo de sensibilidade dos pesos revelaria quais componentes realmente dirigem a busca. A estrutura, no entanto, é reutilizável como está: qualquer problema de escolher e ordenar itens descritos por atributos numéricos contra um perfil, de setlists e cardápios a escalas de aula e sequências de anúncios, cabe no mesmo molde. Os dados e os componentes de qualidade serão reaproveitados como base do algoritmo genético do projeto 4, o que permitirá comparar os dois paradigmas sobre exatamente o mesmo problema.

Projeto 4: Algoritmo Genético

Descrição do Problema

O projeto 4 ataca exatamente o mesmo problema do projeto 3: montar, a partir de um catálogo de cem faixas reais do Spotify com atributos numéricos, uma playlist que atenda a um perfil de usuário — duração aproximada, valores médios desejados de energia, positividade, acústica, dançabilidade e andamento, e um padrão de progressão de energia. Os dados, a função de qualidade, os cinco componentes e os pesos (0,30/0,20/0,25/0,15/0,10) são os mesmos da seção 2.3, de propósito: reusar o problema permite comparar o paradigma da busca local com o paradigma evolutivo sobre exatamente a mesma medida.

O que torna o problema adequado a um algoritmo genético é a estrutura combinatória já discutida: o espaço de candidatos é astronômico e a função de qualidade tem muitos ótimos locais. O Hill Climbing sobe um único pico a partir de um único ponto; o algoritmo genético mantém uma população inteira de soluções, combina pedaços de boas playlists por cruzamento e explora o espaço pela mutação, trocando ótimos locais isolados por uma cobertura ampla da fronteira de boas soluções.

Algoritmo Desenvolvido

Representação: O cromossomo é uma lista de N faixas distintas, com a posição de cada gene significativa: a ordem das faixas define a progressão de energia e a suavidade das transições. O tamanho N é fixo por perfil — estimado pela duração alvo dividida pela duração média do catálogo, resultando em 9 faixas para 30 minutos, 17 para 60 e 5 para 15 — o que viabiliza o cruzamento por permutação sem faixas repetidas. A população inicial é sorteada uniformemente.

Seleção: Torneio de k = 3: três indivíduos sorteados competem e o de maior fitness é copiado para a reprodução. O torneio dispensa normalização do fitness e controla a pressão seletiva apenas pelo tamanho da amostra.

Cruzamento: Order Crossover (OX) com dois pontos de corte, aplicado com probabilidade 0,90: o filho preserva um trecho inteiro do primeiro pai e é completado com os genes restantes na ordem em que aparecem no segundo pai, percorrido circularmente. O resultado é sempre uma permutação válida, o que o cruzamento de um ponto comum não garantiria — ele criaria faixas duplicadas.

Mutação: Com probabilidade 0,10, um dos três movimentos da vizinhança do projeto 3: trocar duas posições, substituir uma faixa por outra do catálogo que não esteja no cromossomo ou inverter um trecho entre dois cortes. Quando o cruzamento não ocorre, o filho é um clone de um dos pais.

Elitismo e parada: Os dois melhores indivíduos passam intactos para a geração seguinte, garantindo que o melhor já encontrado nunca seja perdido. O laço roda até 100 gerações e encerra antes se o melhor fitness não melhorar por 30 gerações consecutivas. Configuração padrão: população 100, cruzamento 0,90, mutação 0,10, elite 2, torneio 3, semente 42, cinco execuções por configuração. Os três perfis repetem as durações e progressões dos perfis do projeto 3, com alvos de atributo de caráter equivalente, e ficam fixados como constantes para tornar as varreduras reproduzíveis.

Resultados

A tabela resume a média de cinco execuções em cada perfil, com os baselines medidos no mesmo processo do algoritmo genético:

Perfil	Aleatória	Gulosa	Genético
30 min, alvos, crescente	0,845	0,884	**0,929**
60 min, alvos, pico	0,816	0,889	**0,937**
15 min, só decrescente	0,842	0,940	**0,984**

O algoritmo genético vence os dois baselines nos três perfis. No perfil de 15 minutos ele chega a 0,984, contra 0,940 do guloso, que nesse perfil se beneficia de uma ordenação perfeita por energia, e a distância para

a aleatória é ainda maior. A precisão na duração confirma que a função de qualidade guia a evolução: as playlists vencedoras fecharam em 30:00, 60:01 e 14:57 para alvos de 30, 60 e 15 minutos. A comparação entre paradigmas, contra os números da seção 2.3, é a seguinte (os baselines foram remedidos dentro do processo do projeto 4, e os alvos de atributo dos dois primeiros perfis são definidos de forma análoga, então os valores são indicativos, não uma comparação controlada):

Perfil	Hill Climbing (proj3)	Simulated Annealing (proj3)	Genético
30 min, alvos, crescente	0,892	0,916	**0,929**
60 min, alvos, pico	0,910	0,907	**0,937**
15 min, só decrescente	0,970	0,981	**0,984**

A evolução supera as duas buscas locais nos três perfis, com a maior vantagem nos dois primeiros, onde o espaço é maior e a população explora frentes variadas enquanto o Hill Climbing estaciona no primeiro ótimo local. A playlist vencedora do perfil decrescente abre com Não é Pressa, é Pressão (Rincon, Ana Castela e DJ BOSS), passa por Hard Times e Algo Parecido, e desce até Enemy e Silencio: cinco faixas cujas energias caem degrau a degrau, totalizando 14:57. A vencedora do perfil de pico fecha em 60:01 com dezessete faixas e a curva pedida desenhada com precisão.

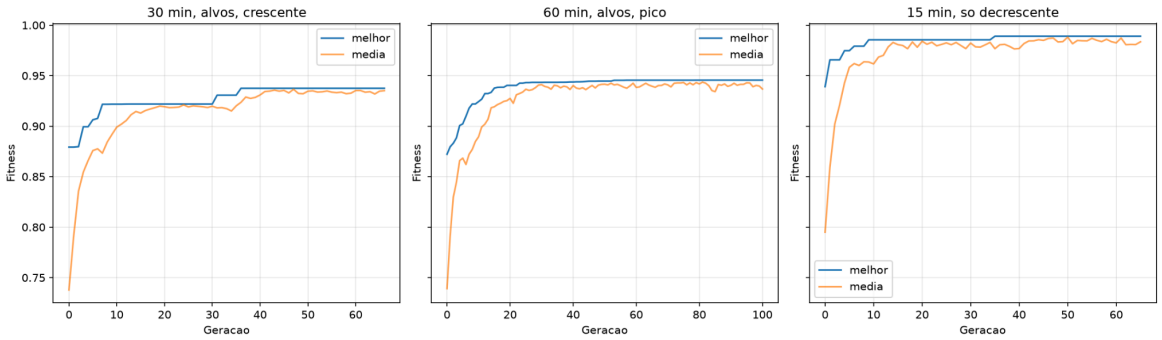


Figura 7 - convergência do melhor e da média de fitness por geração nos três perfis. O melhor sobe em degraus a cada melhoria aceita e a média o acompanha de perto, refletindo a pressão seletiva do torneio e o efeito nivelador do elitismo.

As varreduras de parâmetros, com média geral sobre os três perfis, estão abaixo:

População	20	50	100	200
Média geral	0,945	0,951	0,954	0,955

Taxa de mutação	1%	5%	10%	20%
Média geral	0,944	0,950	0,951	0,953

Os dois parâmetros mostram retornos decrescentes: dobrar a população de 100 para 200 rende apenas um milésimo, e a mutação alta não degenera a busca, sinal de que o elitismo estabiliza a evolução. Uma execução do algoritmo genético leva cerca de 0,7 segundo e o experimento completo, com varreduras, dois minutos.

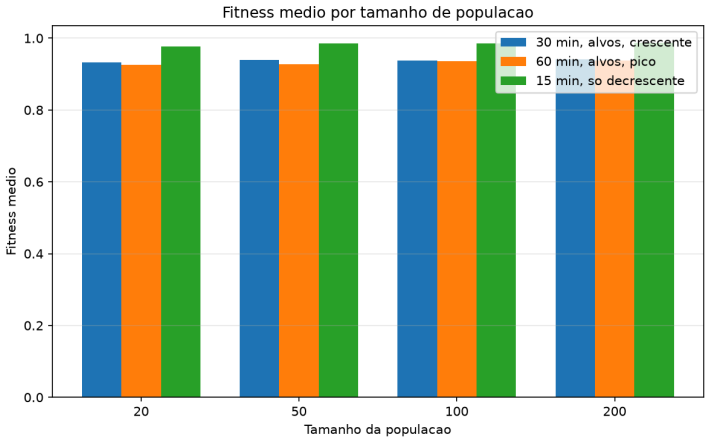


Figura 8: fitness médio por tamanho de população, agrupado por perfil.

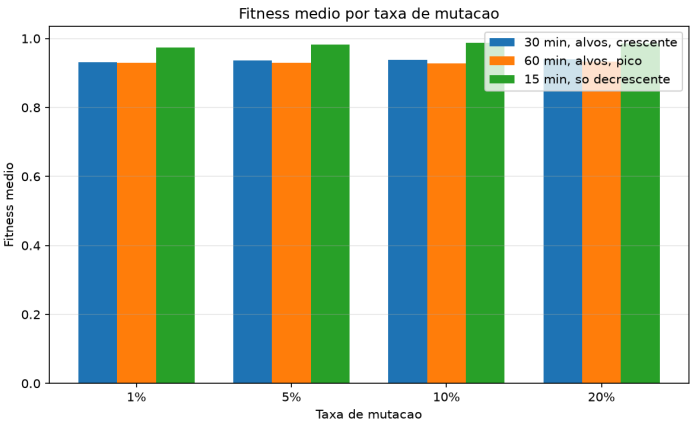


Figura 9: fitness médio por taxa de mutação, agrupado por perfil.

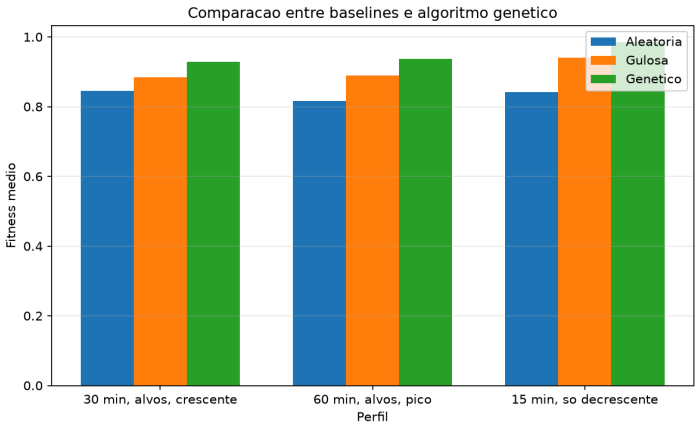


Figura 10: comparação entre baselines e algoritmo genético, agrupada por perfil.

Considerações Finais

O projeto mostra que um algoritmo genético elementar, com operadores de livro-texto, resolve melhor que a busca local o mesmo problema de composição de playlists: venceu o Hill Climbing e o Simulated Annealing nos três perfis e se mostrou robusto à escolha de população e taxa de mutação, o que reduz a necessidade de calibração fina que o SA exigiu no projeto 3. O potencial vem da exploração paralela: enquanto a busca local concentra todo o orçamento num único caminho, a população cobre regiões diferentes do espaço e o cruzamento recombina os blocos que funcionam.

As limitações são as mesmas da seção 2.3, com acréscimos evolutivos. A função de qualidade continua sendo uma combinação linear de pesos arbitrados à mão, e a resposta final é tão boa quanto eles. O comprimento fixo N impede playlists que precisariam de uma faixa a mais ou a menos para encaixar a duração exata, e a população grande torna o algoritmo mais caro que o Hill Climbing: 10 mil avaliações por execução contra algumas dezenas. A evolução também é estocástica por natureza, e a substituição herda a dependência da ordem de conjuntos do Python, já discutida na seção 2.3, o que exige PYTHONHASHSEED fixo para reprodução exata.

Como melhorias, a mutação ganharia movimentos de inserção e remoção, hoje ausentes, o comprimento poderia ser tratado por dois cromossomos paralelos, e taxas adaptativas ou um esquema de ilhas diversificariam a exploração. A comparação com o projeto 3, no entanto, já cumpre o objetivo: sobre o mesmo problema e a mesma função, o paradigma populacional superou o paradigma da busca local, e a estrutura do código permite repetir o experimento para qualquer problema de escolher e ordenar itens contra um perfil numérico.

Projeto 5: CSPs — Satisfação de Restrições

Descrição do Problema

Todo semestre a coordenação da FGA precisa alocar as turmas ofertadas a horários e salas. Este projeto modela a oferta real de Engenharia de Software do 2º/2026, com 99 turmas do fluxo, e monta a grade como um problema de satisfação de restrições. Cada encontro de turma escolhe um par formado por bloco de horário e sala, e as exigências são binárias entre encontros: o professor não pode estar em dois lugares ao mesmo tempo, duas turmas não podem dividir a mesma sala no mesmo horário e a sala precisa comportar as vagas da turma.

O problema é adequado a CSP por dois motivos. O espaço de combinações cresce além de qualquer enumeração, com centenas de pares bloco-sala por encontro, e a solução é uma atribuição completa e consistente, não um caminho. Duas restrições que parecem naturais foram testadas como cenários adicionais, e os dados reais as desmentiram: as turmas dos Projetos Integradores se encontram todas no mesmo horário, em salas diferentes, e o terceiro e o quarto períodos do fluxo têm mais encontros semanais que blocos disponíveis, o que torna a separação por período impossível por simples contagem. Esse contraste entre o modelo imaginado e o modelo que os dados sustentam é parte do resultado do projeto.

Algoritmo Desenvolvido

Cada encontro de turma vira uma variável cujo domínio são os pares (bloco, sala) filtrados pela capacidade, usando os blocos de duas aulas da notação do SIGAA e um conjunto de 26 salas com capacidades sintéticas, já que a lotação real não é pública. O grafo de restrições guarda apenas as relações fortes, que proíbem o compartilhamento de bloco; a unicidade da sala é conferida na atribuição com um índice de salas ocupadas por bloco, o que mantém o grafo esparso sem perder correção.

A busca parte de um backtracking simples e acumula recursos em cinco variantes: a heurística MRV de menor domínio restante, a ordenação LCV pelo efeito de poda, o forward checking, que poda os vizinhos a cada atribuição, e o MAC, que mantém a arco consistência com AC-3 propagando remoções entre vizinhos. Todas as métricas contam nós, atribuições, retornos e podas, e a solução final passa por uma verificação independente que percorre os pares de variáveis conferindo cada restrição, como nos projetos anteriores. A inviabilidade do cenário de períodos é provada por um argumento de casa dos pombos, sem busca.

Resultados

A instância real tem 304 variáveis. Todas as variantes terminam em menos de dois segundos, e a tabela resume os retornos nos dois cenários resolvíveis.

Variante	Cenário A (retornos)	Cenário B (retornos)
Backtracking puro	30.548	37.359
MRV	32.420	38.972
MRV+LCV	32.420	38.972
MRV+LCV+FC	19.935	8.989
MRV+LCV+MAC	7.850	4.914

Três leituras se impõem. A inferência é o que realmente paga: o MAC reduz os retornos em quatro vezes no cenário A e em sete e meia no cenário B. O MRV sozinho chega a piorar o caminho da busca, porque escolher bem a variável sem escolher bem o valor apenas troca a ordem das tentativas. E o LCV não age sem inferência: com domínios cheios, todos os valores têm o mesmo impacto de poda, e a ordenação fica neutra, o que explica as colunas idênticas de MRV e MRV+LCV. O cenário C foi declarado infactível antes da busca: o período 3 concentra 78 encontros semanais e o período 4 concentra 62, ambos acima dos 54 blocos disponíveis. No experimento de escala, com instâncias sintéticas de 20, 40 e 60 turmas, os retornos do backtracking puro crescem de 2.460 para 17.604 enquanto o forward checking e o MAC permanecem na casa de 46 a 195.

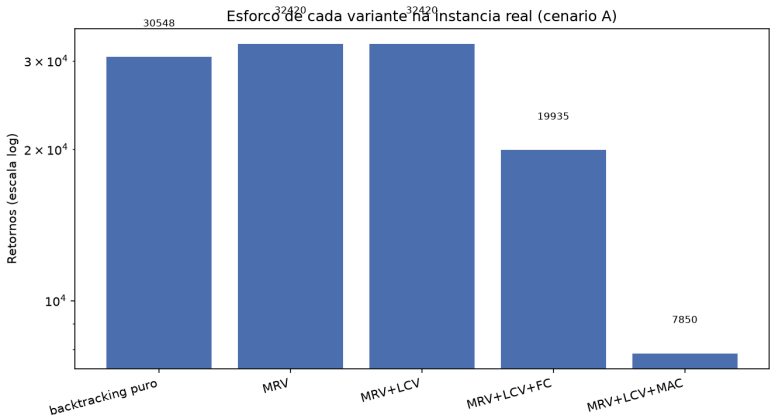


Figura 7: retornos de cada variante na instância real do cenário A, em escala logarítmica, com as barras em azul quando a busca completou.

Considerações Finais

O resultado central é que, neste domínio, propagação de restrições vale mais que heurísticas de escolha isoladas, e o MAC entrega a maior redução de esforço sem perder a completude da busca. As limitações são conhecidas: as salas têm capacidades sintéticas, o bloco de duas aulas é a unidade mínima e preferências de conforto, como evitar janelas na grade dos professores, ficaram fora do escopo. A estrutura, porém, transfere-se diretamente para qualquer alocação de recursos com exclusividade mútua, de escalas de plantão a distribuição de salas de prova, e o contraste entre as restrições hipotéticas e as reais mostrou que modelar com dados vale tanto quanto escolher bem o algoritmo.

Projeto 6: Banco de Conhecimento

Descrição do Problema

O projeto 6 fecha o ciclo do mesmo problema — montar playlists sobre o catálogo de cem faixas reais do Spotify — com o terceiro paradigma da disciplina: em vez de otimizar uma função numérica como nos projetos 3 e 4, o sistema trata a recomendação como um problema de **inferência simbólica**. Uma base de conhecimento guarda fatos sobre as faixas e sobre o usuário, regras de produção no formato de cláusulas de Horn ligam o contexto do usuário a exigências musicais, e motores de encadeamento derivam as recomendações.

O que justifica um banco de conhecimento aqui é a **explicabilidade**. Um filtro numérico devolve uma lista; um sistema simbólico devolve a lista e a cadeia de regras que justifica cada faixa ("esta faixa entrou porque academia exige energia alta, e ela tem energia 0,81"). Essa capacidade de explicar o próprio raciocínio é a contribuição central do paradigma e o que o diferencia dos projetos anteriores no portfólio.

Algoritmo Desenvolvido

Representação. Os fatos da base empregam notação de lógica de primeira ordem aterrada, na forma de predicados com argumentos. Predicados de posse associam cada faixa a propriedades como energia alta ou andamento rápido; predicados de contexto descrevem a ocasião, o humor, o período e a duração alvo informados pelo usuário; predicados de exigência representam as exigências musicais derivadas pela inferência. Os atributos numéricos do catálogo são convertidos em predicados por limiares fixados no código, com energia e positividade divididas em terços, andamento segmentado em torno de 95 e 120 batidas por minuto, e acústica e dancabilidade separadas pela metade da escala.
Regras. Treze regras de preferência relacionam ocasião, humor e período às exigências musicais, com prioridades que codificam a importância relativa de cada fonte: a ocasião recebe prioridade 3, o humor 2 e o período 1. Assim, a ocasião academia deriva as exigências de energia alta e andamento rápido, a ocasião festa deriva dancabilidade e positividade alta, e a ocasião estudo deriva acústica. As regras de recomendação são instanciadas para cada par de faixa e predicado, associando a exigência correspondente à propriedade observada na faixa, o que totaliza 813 regras aterradas, estrutura equivalente a uma base proposicional obtida a partir de uma descrição em primeira ordem.
Encadeamento progressivo. O motor de inferência percorre a lista de regras repetidamente até atingir o ponto fixo, disparando toda regra cujos antecedentes estejam presentes na base e cujo consequente ainda não tenha sido derivado. Cada derivação registra a regra que a produziu, informação que sustenta a etapa de explicação. Nas execuções realizadas, o ponto fixo é atingido em milissegundos, com 24 a 146 fatos derivados por cenário.
Encadeamento regressivo e explicação. A explicação de uma recomendação consiste em percorrer, em sentido contrário, a regra que derivou o fato correspondente. A prova da primeira faixa do cenário academia, por exemplo, parte do fato de contexto que informa a ocasião academia, aplica a regra que deriva a exigência de andamento rápido e encerra no fato que atribui andamento rápido à faixa recomendada. Esse procedimento devolve, para cada faixa, uma árvore de prova que justifica sua inclusão na playlist.
Seleção e ordenação. Cada faixa recebe a soma das prioridades das exigências que atende; as faixas de maior pontuação entram na playlist até que a próxima ultrapasse a duração alvo, e a lista final é ordenada pelo padrão de progressão do cenário, reutilizando o procedimento do projeto 3. Nos casos em que o conhecimento deriva exigências conflitantes, como a festa que pede positividade alta e o período noturno que pede positividade baixa, a prioridade decide o resultado tanto na seleção quanto na métrica de comparação.

Resultados

A tabela resume a inferência em cada cenário de usuário:

Cenário	Regras instanciadas	Regras disparadas	Fatos derivados	Tempo	Playlist
Academia, 60 min, crescente	813	146	146	0,08 s	13 faixas, 59:14
Estudo, 45 min, constante	813	24	24	0,02 s	12 faixas, 44:28
Festa, 90 min, pico	813	138	138	0,06 s	23 faixas, 89:39

A comparação com os baselines usa duas medidas: a **cobertura**, fração das exigências atendidas pela playlist (ponderada por prioridade), e a **distância média aos alvos numéricos** de cada exigência, a mesma tradução usada pelos baselines:

Cenário	Cobertura Regras	Cobertura Gulosa	Cobertura Aleatória	Distância Regras	Distância Gulosa	Distância Aleatória
Academia	**1,000**	0,941	0,694	0,129	**0,061**	0,179
Estudo	**0,771**	0,725	0,134	0,141	**0,115**	0,325
Festa	**0,658**	0,523	0,445	0,207	**0,147**	0,231

As regras vencem em cobertura nos três cenários — no estudo, 0,771 contra 0,134 da aleatória — porque otimizam exatamente o critério simbólico. O guloso vence na distância numérica, o que é honesto: ele persegue diretamente a métrica contínua que as regras apenas aproximam por limiares. O sistema de regras fica sempre à frente da aleatória e perto do guloso, com a vantagem de explicar cada escolha. O cenário da festa ilustra o conflito resolvido por prioridade: a ocasião exige positividade alta e dancabilidade, o período noturno pede positividade baixa e acústica, e as prioridades 3 a 1 dão a vitória à festa.

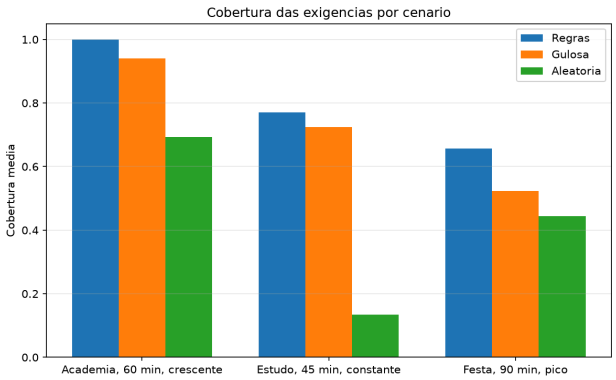


Figura 11: cobertura das exigências por cenário para o sistema de regras e os baselines.

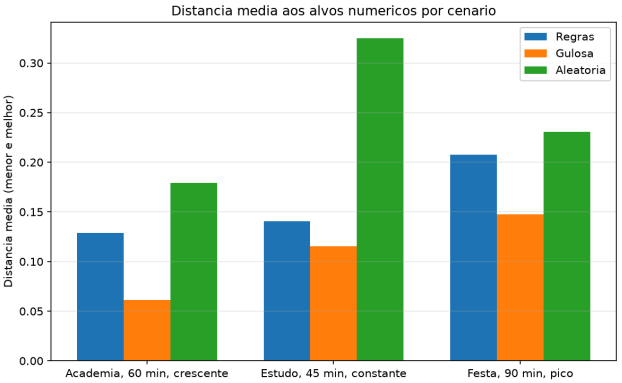


Figura 12: distância média aos alvos numéricos por cenário (menor é melhor). O guloso lidera por otimizar a métrica diretamente; as regras superam a aleatória em todos os cenários.

Considerações Finais

O projeto demonstra que uma base de conhecimento com 813 regras aterradas responde em milissegundos e entrega aquilo que a otimização não entrega: uma playlist explicável, na qual cada faixa possui justificativa auditável na forma de árvore de prova. O motor de inferência é genérico, pois os fatos e as regras poderiam descrever outros domínios, como diagnósticos médicos, escalas de aula ou cardápios, e é extensível, uma vez que acrescentar conhecimento novo se resume a acrescentar regras, sem reescrever a lógica de busca.

As limitações observadas são as do paradigma simbólico clássico. O conhecimento é estático e os limiares que convertem números em predicados são arbitrários, o que explica a desvantagem diante do guloso na métrica contínua. Cláusulas de Horn sem negação não expressam restrições como a proibição de misturar determinados estilos, e o sistema não trata incerteza nem preferências graduais. Como melhorias, a base poderia incorporar lógica de primeira ordem com quantificadores e unificação, fatores de confiança fuzzy em substituição às prioridades fixas e, na direção apontada pelo portfólio, um modelo híbrido que empregue as regras para restringir o espaço de busca e a função de qualidade do projeto 4 para otimizar dentro dele, somando a explicabilidade do simbólico à precisão do numérico.

3 Considerações Finais Gerais

Os seis projetos percorrem os quatro grandes blocos dos métodos clássicos e, lidos em conjunto, respondem a uma pergunta única: qual paradigma usar em cada situação. A resposta que os experimentos sustentam é que a escolha depende menos da qualidade genérica de um algoritmo e mais da estrutura do problema e do que se exige da resposta.

Os dois primeiros projetos delimitam o valor da informação. No projeto 1, a heurística geométrica quase não ajudou, com razão h/h* de 0,445, e a heurística perfeita mostrou o teto do que o conhecimento de domínio pode render: trinta e duas vezes menos expansões. No projeto 2, a ausência de informação foi levada ao limite, e a prova de que as oitenta consultas são provavelmente ótimas mostrou que o gargalo era de modelagem, não de algoritmo, com o UCS inaplicável não por lentidão, mas por formalização errada. Juntos, os dois projetos ensinam que a informação compensa quando é boa e que a busca cega brilha quando não há o que saber.

Os projetos 3, 4 e 6 formam o arco mais revelador do portfólio, porque atacam o mesmo problema com três paradigmas. A busca local resolve em centésimos de segundo, desde que calibrada; a população do algoritmo genético venceu as duas buscas locais nos três perfis sem exigir calibração fina, cobrindo frentes que um único caminho não alcança; e o sistema simbólico perdeu para o guloso na métrica contínua, mas foi o único a explicar cada escolha. O mesmo problema revelou assim a vocação de cada paradigma: otimizar rápido, otimizar melhor e explicar.

O projeto 5 acrescenta uma lição sobre modelagem que ecoa em todo o relatório. Duas restrições que pareciam naturais foram desmentidas pelos dados reais, uma por inspeção e outra por prova de contagem, e a propagação de restrições mostrou valer mais que heurísticas de escolha isoladas, com o MAC reduzindo em até sete vezes os retornos. A verificação independente, presente em todos os projetos, impediu que cada algoritmo assinasse o próprio trabalho, e o uso de dados reais, do OpenStreetMap ao SIGAA, transformou intuições em números.

A relevância dos métodos clássicos permanece porque eles são a camada de garantia da inteligência artificial moderna. Roteadores e navegadores continuam a ser A* e variações, resolvedores com propagação sustentam escalas de horários e compiladores, algoritmos evolutivos seguem na otimização de projetos e a exigência crescente de sistemas explicáveis devolve importância à inferência simbólica. Os modelos contemporâneos são superiores em percepção e linguagem, mas não substituem essas garantias: quem precisa de ótimo, de prova ou de justificativa recorre aos clássicos, que este portfólio exercitou, mediu e, quando os dados pediram, corrigiu.

Referências Bibliográficas

[1] RUSSELL, S.; NORVIG, P. Artificial Intelligence: A Modern Approach. 4. ed. Pearson, 2022.

[2] CORMEN, T. H. et al. Introduction to Algorithms. 4. ed. MIT Press, 2022.

[3] SOARES, Fabiano Araujo. FGA 0221 – Inteligência Artificial: Parte II – IA Simbólica ou Good Old-Fashioned Artificial Intelligence (GOFAI) – Resolvendo problemas por busca. Brasília: Faculdade de Ciências e Tecnologias em Engenharia, Universidade de Brasília, 2025. 31 slides. Apresentação de slides utilizada na disciplina FGA 0221 – Inteligência Artificial.

[4] SOARES, Fabiano Araujo. Resolvendo problemas por busca: Ambientes complexos. Brasília: Universidade de Brasília, 2025. Slides apresentados na disciplina FGA0221 – Inteligência Artificial, Faculdade de Ciências e Tecnologias em Engenharia.

[5] SOARES, Fabiano Araujo. FGA 0221 – Inteligência Artificial: Parte II: IA Simbólica – Problemas de Satisfação de Restrições. Brasília: Faculdade de Ciências e Tecnologias em Engenharia, Universidade de Brasília, 2025. 34 slides. Apresentação em slides.

[6] GDEVALVES. AI-Portfolio. Arquivo: proj3_ajuste_sa.md. GitHub, 2026. Disponível em: https://github.com/GdevAlves/AI-Portfolio/blob/main/parte_II/results/proj3_ajuste_sa.md. Acesso em: 9 out. 2026.